СОДЕРЖАНИЕ

СОДЕРЖАНИЕ 1
Понятие диаграмм 2
Почему UML? 3
Каковы преимущества UML? 3
Типы диаграмм UML 4
Поведенческие диаграммы 4
Структурные диаграммы 5
Требования к диаграммам 5
Диаграмма вариантов использования(use-case) 6
Описание элементов диаграммы прецедентов 7
Уточнение диаграммы прецедентов 10
Практическая работа №8. 15

Понятие диаграмм

Наша цель – научиться рассматривать поставленную задачу со всех точек зрения. Зачем это нужно?

Предположим, мы поставили себе цель разработать БД для работы с кофейней. Разрабатываемая структура может включать в себя сотрудников: их ФИО, адреса проживания, номера телефонов, годы рождения, стаж, пенсию, девичью фамилию кошки, и, конечно же, все логины и пароли от социальных сетей…

Однако, при непосредственной работе с сотрудником, мы понимаем, что большинство информации нам было и не нужно, а та, что должна была быть – не предусмотрена. Именно для этого необходимо продумывать алгоритмы и проектировать диаграммы.

Другими словами, диаграммы необходимы для полного описания масштабных работ, с целью уточнения структуры проекта.

UML-диаграммы

В качестве диаграмм мы понимаем не гистограммы и графики. В нашем случае, мы рассматриваем UML-диаграммы.

UML-диаграмма — это схема, нарисованная с применением символов **UML**. Она может содержать множество элементов и соединений между ними. Элементами **диаграммы UML** могут быть классы программного кода, страницы сайта, части механизма, зоны торгового зала — в зависимости от того, какой процесс или какую сущность описывает её создатель. UML помогает продемонстрировать возможные ошибки в структурах приложений, поведении системы и других бизнес-процессах.

ВАЖНО*: посвящу отдельный абзац **бизнес-процессам**. Очень важно понимать, что мы в рамках бизнес-процесса понимаем совершенно любое действие и ситуацию. Скажем, мы можем описать с помощью диаграммы работу программного обеспечения, или увольнение Дмитрия Александровича, или отчисление Коростелевой Киры или то, как вы с утра выгуливали собаку. Как раз возможность дать трактовку нескольким сферам

жизни одним графическим представлением называется **описание бизнес-процесса**. А важно это, потому что исходя из этого складываются типы диаграмм.

Почему UML?

Впервые UML появился еще в 1990-х годах благодаря трем инженерам-программистам — Грэди Бучу, Ивару Джекобсону и Джеймсу — поскольку они хотели разработать менее хаотичный способ представления разработки все более сложного программного обеспечения, в то же время отделяя методологию от самого процесса. Сегодня UML по-прежнему является стандартной практической нотацией для разработчиков, а также для руководителей проектов, владельцев бизнеса, технических предпринимателей и специалистов из разных отраслей.

Каковы преимущества UML?

- упрощает сложности;
- сохраняет возможности открытого общения;
- автоматизирует производство программного обеспечения и процессов;
- помогает решить постоянные проблемы с архитектурой;
- улучшает качество работы;
- сокращает затраты и время выхода на рынок.

Типы диаграмм UML

Существует два основных типа диаграмм UML**: структурные диаграммы** и **поведенческие диаграммы** (а внутри этих категорий имеется много других). Эти варианты существуют для представления многочисленных типов сценариев и диаграмм, которые используют разные типы людей.

От заказчиков и руководителей проектов до технических писателей, конструкторов, аналитиков, программистов и тестеров — представители каждой роли будут использовать конкретную диаграмму в соответствии со своими потребностями. Это означает, что каждый шаблон требует различного фокуса и уровня детализации. **Цель UML** — визуально представить диаграммы, которые легко понять каждому.

Поведенческие диаграммы

Основное внимание здесь уделяется динамическим аспектам системы программного обеспечения или процесса. Эти диаграммы показывают функциональные возможности системы и демонстрируют, что должно происходить в моделируемой системе.

Говоря проще, поведенческие диаграммы описывают **поведение** объектов или субъектов. В этом типе нет строгого упора на сухую разработку ПО. При разработке поведенческой диаграммы ВСЕГДА можно дать описание бизнес-процессу.

Еще проще – есть актер = поведенческая диаграмма.



Рисунок 1 – Какая-то поведенческая диаграмма

Структурные диаграммы

Структурные диаграммы представляют статическую структуру программного обеспечения или системы, они также показывают различные уровни абстракции и реализации. Они используются, чтобы помочь визуализировать различные структуры, составляющие систему, например, базу данных или приложение. Они показывают иерархию компонентов или модулей и то, как они связаны и взаимодействуют между собой. Эти инструменты обеспечивают руководство работы и гарантируют, что все части системы функционируют так, как задумано по отношению ко всем остальным частям.

Само название говорит о том, что диаграммы данного типа будут передавать **структуру** программы. С помощью таких диаграмм уже нельзя дать описание бизнес-процессу. Они узконаправленны и необходимы лишь для того, чтобы дать описание чего-то информационного.

Требования к диаграммам

Как к программному продукту, так и к унифицированному языку моделирования (UML) предоставляются свои требования:

- диаграмма должна точно соответствовать поставленной задаче;
- четкость и ясность модели;
- вся диаграмма (комплекс диаграмм) должна быть выполнена в одном стиле;
- в случае, если одной диаграммы недостаточно для сбора достоверных данных о разработке, то допустимо прибегнуть к разработке другой – дополняющей модели.

Диаграмма вариантов использования(use-case)

Давайте пойдем по порядку и начнем рассматривать группу поведенческих диаграмм. Самым ярким и простым примером является **диаграмма вариантов использования**, она же **диаграмма прецедентов**, она же **use-case**. Запоминайте, потому что на экзамене потребуются все названия! А названий она имеет так много (как и многие другие диаграммы) из-за неточностей перевода.

В этом типе описывается, что делает система, но не то, как она это делает. **Вариант использования** — это набор событий, которые происходят, когда “оператор” использует систему для завершения процесса*. Оператор определяется как кто-либо или что-либо, взаимодействующее с системой (человек, организация или приложение) из-за пределов системы. Таким образом, диаграмма вариантов использования визуально описывает этот набор последовательностей и представляет функциональные требования системы.

Примечание*: Диаграмма вариантов использований описывает систему вне временного параметра. То есть, все ветки происходят и не происходят одновременно.

Описание элементов диаграммы прецедентов

Первое, что нужно сделать при создании диаграммы вариантов использования – это определить **действующие лица (субъекты, актеры)**. В качестве «действующих лиц» или же «актеров», могут выступать как живые люди, так и неодушевленные объекты, способные побуждать к действию.

Отличительной чертой данной поведенческой диаграммы является то, что ВСЕ действующие лица будут представлены, как «человечек» (он же актер или субъект).



Рисунок 3 – Представление актеров.

Дальше, мы должны определить **действия**, они же **варианты использования,** они же **прецеденты** (так вот почему она так называется!). Чтобы это сделать, мы должны подробно изучить предметную область поставленной задачи. Начать стоит с масштабного описания.

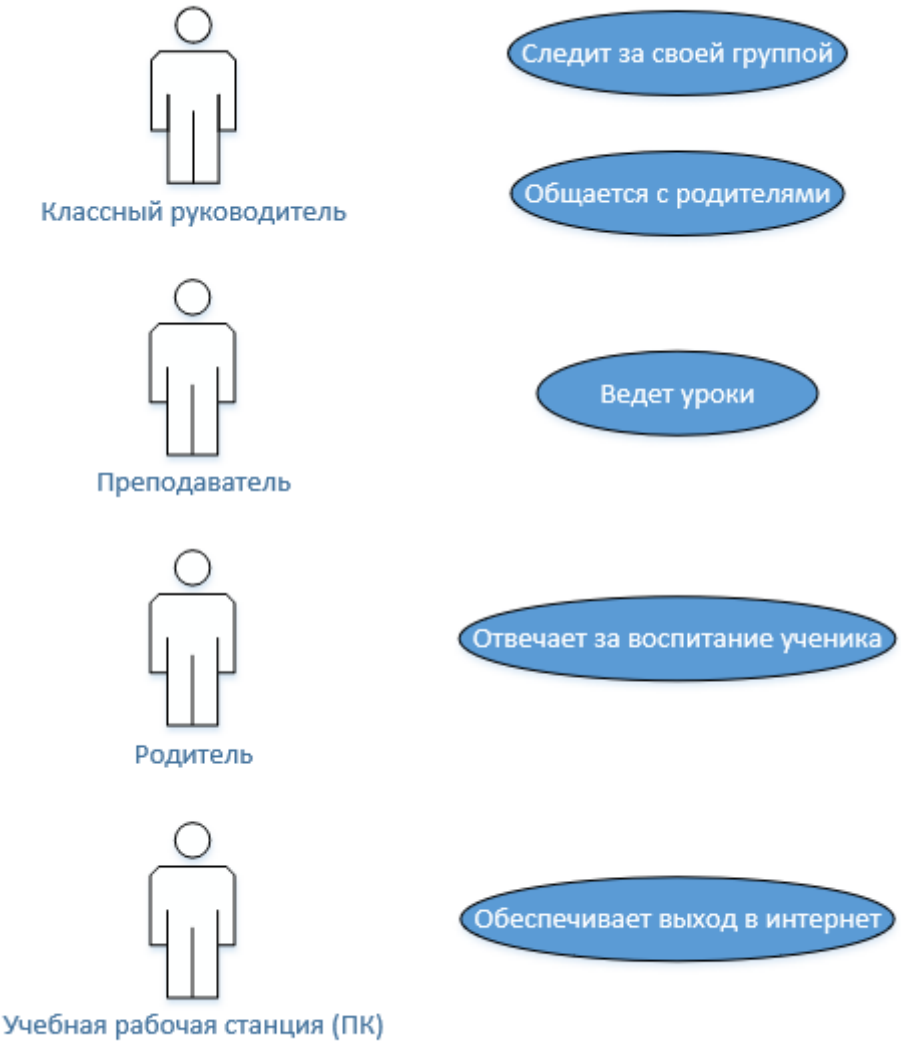


Рисунок 4 – Определение вариантов использования.

Чтобы не запутаться, кому из актеров какие действия принадлежат, мы используем связи. Например, **связь ассоциация** – это связь которая имеет ненаправленный непрерывный вид, соединяющий актеров с их действиями*.

Или, взять, к примеру, **связь зависимость** – это вид связи, который имеет тот же вид, но соединяет действие и актера. (Ассоциация – когда

побудителем события является актер. Зависимость – побудитель - действие). Но, так как визуально они не различаются, разница будет лишь в теории.

Примечание*: Ассоциация и зависимость соединяют исключительно актера и вариант использования, для соединения однотипных объектов используются другие связи (См. уточнение вариантов использования).

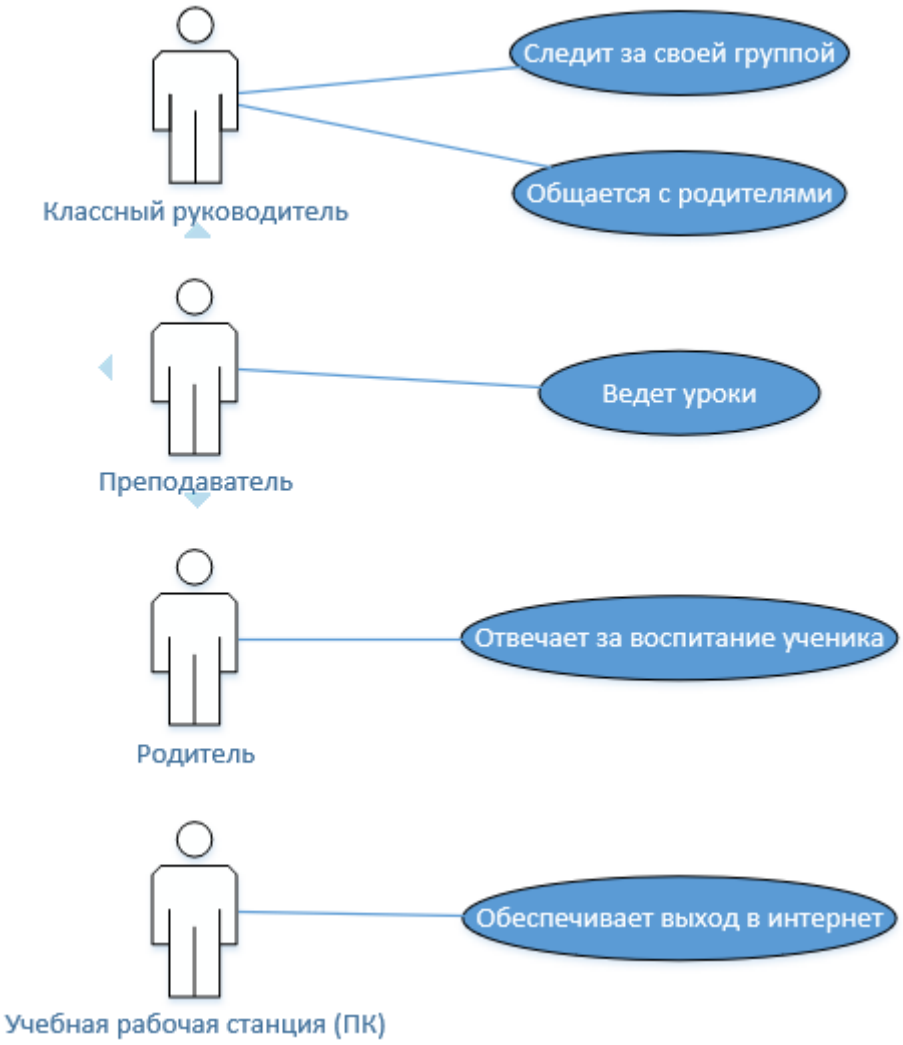


Рисунок 5 – Расстановка связей.

Уточнение диаграммы прецедентов

Когда мы умеем делать диаграмму вариантов использования в общем виде, мы должны научиться работать с уточнениями и всеми тонкостями.

Начнем с **подсистемы** или, как ее еще называют, **предметной области**. Благодаря подсистеме мы объединяем совокупность действий* и даем обобщенное название исследуемой структуры.

Вообще, создавая диаграммы прецедентов - стоит начать с предметной области, дабы определить направление разработки и структурировать данные по классам, таблицам и д.р. В нашем же случае, используется одна предметная область, которую при необходимости можно опустить.

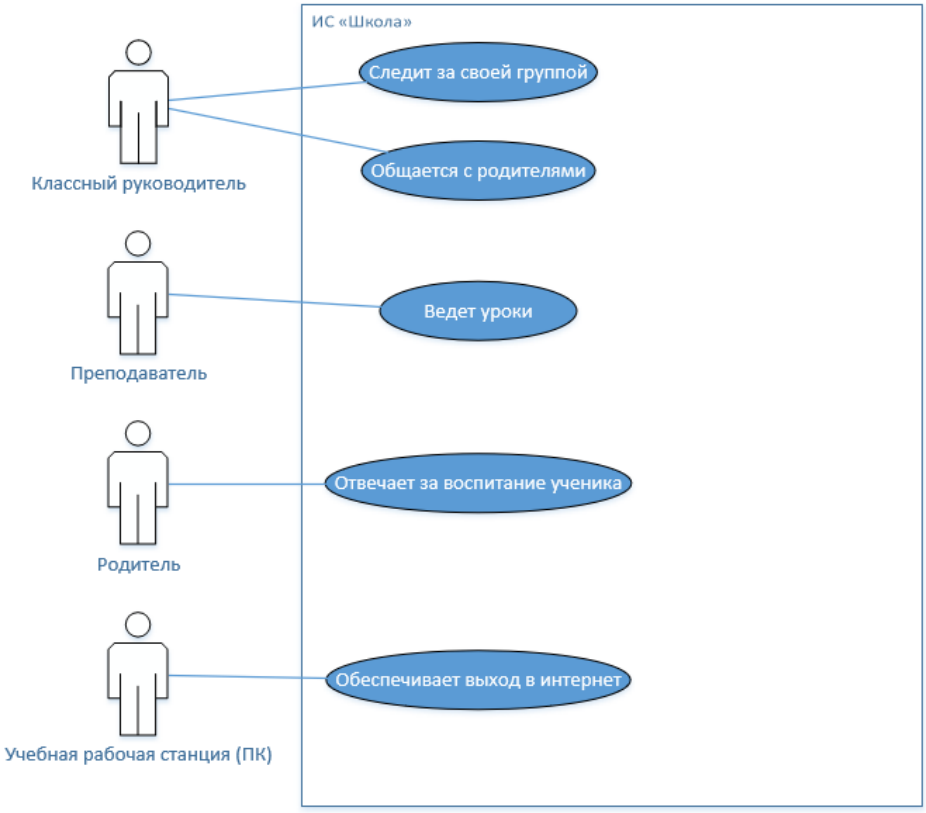


Рисунок 6 – Пример использования предметной области.

Примечание*: также, важно при создании предметной области участие исключительно вариантов использования.

Предположим, у нас существует баба Зина, которая работает на благо библиотеки. Ее действия направлены на улучшение предметной области, но

если она уволится, то библиотека не рухнет из-за этого. Поэтому актеры-сотрудники – никогда не входят в подсистему.

Однако, если актер неодушевленный и необходим для сохранения работы предметной области, то он может быть включен в подсистему.

Как пример, можем рассмотреть работу заправки: у нас существует заправщик, бензоколонка, автомобиль и автовладелец. Далее рассуждаем:

Заправщик – заменим; автовладелец – сегодня один, завтра другой – заменим; автомобиль – тоже заменим (потому что только один Опель василькового цвета не может стоять на заправке до скончания веков и жрать бензин); и остается только разобраться с бензоколонкой – а вот она то как раз уже является неотъемлемой частью, в предметную область – включить!

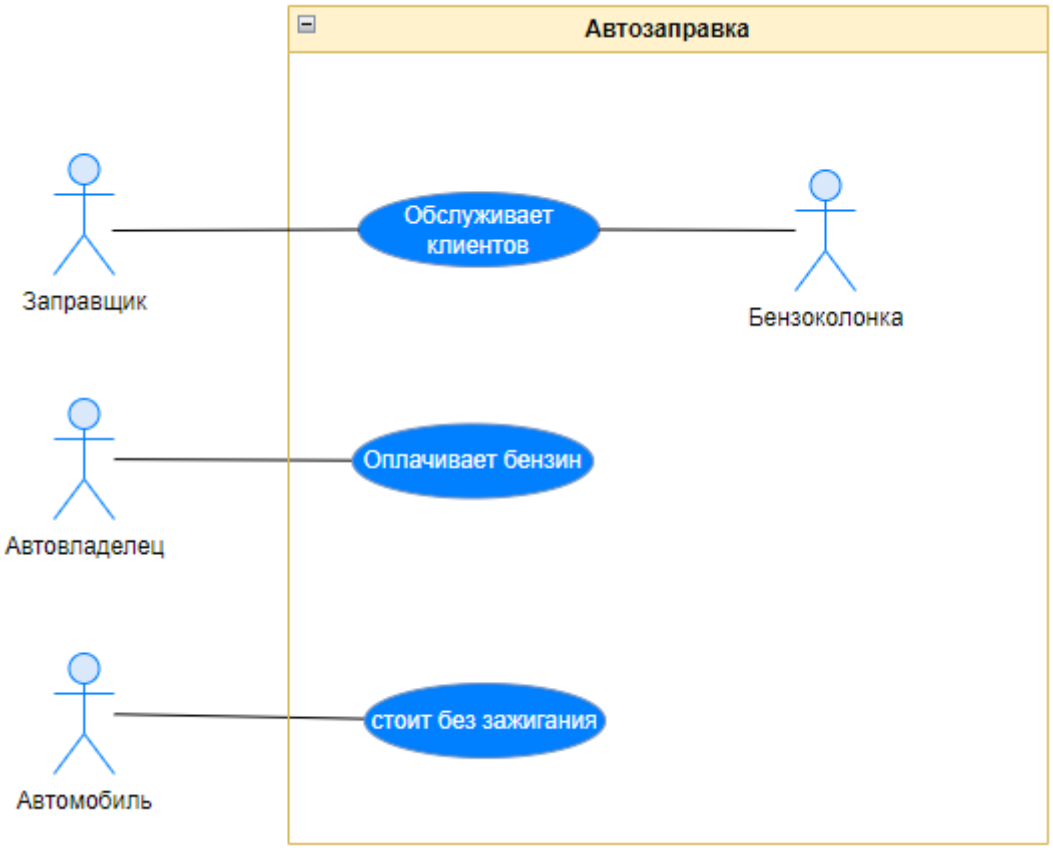


Рисунок 7 – Уточнение предметной области.

Диаграмма вариантов использования - одна из самых простых. У нас есть действия, есть актеры, есть связи между ними. Однако, бывают ситуации, когда нам нужно будет сделать связь «актер => актер».

Немного вернемся - к рисунку 6. У нас есть Классный руководитель и обычный преподаватель, но если подумать, то классный руководитель выполняет те же функции, что и преподаватель. Вместо того, чтобы много раз прописывать однотипные действия, мы можем обобщить классного руководителя в преподавателя. Тем самым мы отобразим, что все преподаватели ведут уроки, но не все общаются с родителями.

Для такого отображения в диаграмме используют связь «**обобщение**». Она имеет непрерывную линию со стрелкой*.

Примечание*: Стрелка будет направлена от частного случая к общему.

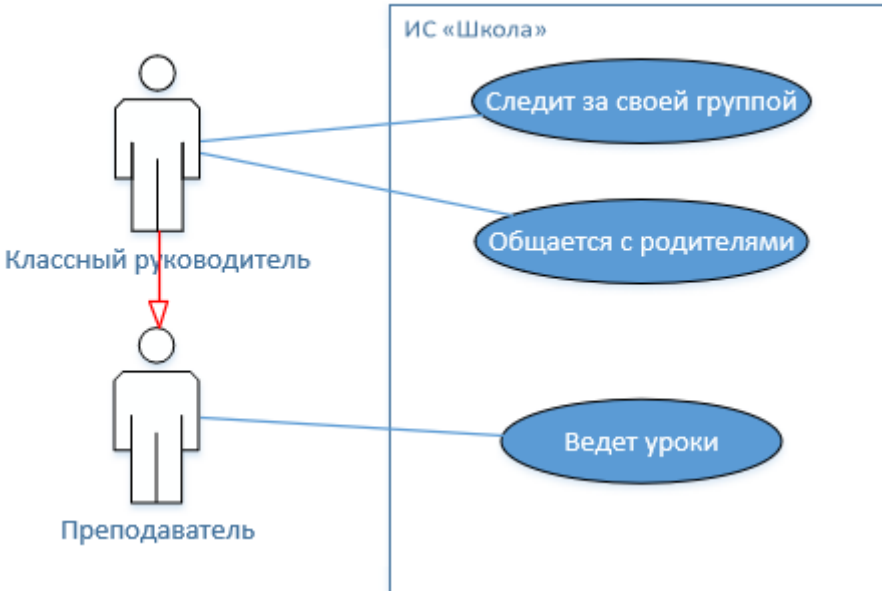


Рисунок 8 – Уточнение обобщения.

Для изучения связей между действиями, подробно рассмотрим преподавателя и спектр его обязанностей. Каждый преподаватель ведет лекции, выставляет оценки и проверяет практику.

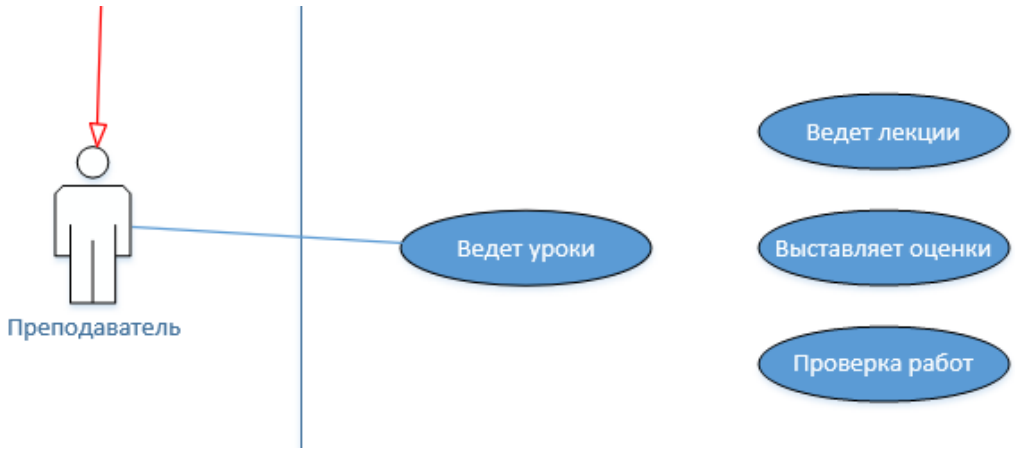


Рисунок 9 – Подробное изучение возможностей преподавателя.

Теперь давайте рассуждать: Проведение урока не может пройти без лекции. Значит, «Ведет лекции» — это составляющая часть от урока. Следовательно, в «Ведет уроки» мы **включаем** «Ведет лекции». А вот «Выставляет оценки» - необязательная часть пары. Получается, что «Выставление оценок» **расширяет** «Ведение урока».

Для отображения типа «***включения***» и «***расширения***» используются соответственные одноименные связи. Сами по себе они представляют прерывающиеся прямые со стрелками. Разница будет в направлении стрелок и надписях*. Включение – это связь, которая подразумевает фундаментальную принадлежность к общему действию, расширение же – расширяет общее действие.

Примечание*: Включение направлено от общего случая к частному. (Общее действие **включает** частное).

Расширение направлено от частного случая к общему. (Побочное действие **расширяет** общее).

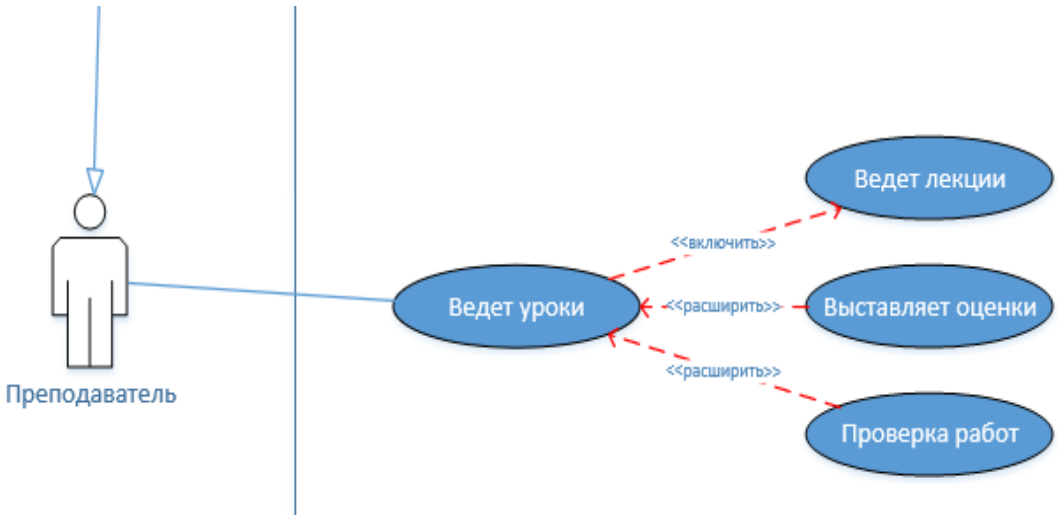


Рисунок 10 – Связи между действиями.

Практическая работа №8.

Задание 1.

1. Вспомнить работу с графическим редактором Draw.io для создания алгоритмов и диаграмм.

2. Изучить теоретический материал по созданию диаграммы вариантов использования.

3. Разработать диаграммы, согласно требованиям.

4*. Каждая диаграмма должна быть выполнена в одном документе, но разных страницах.

Части (для всех).

1. Создать диаграмму, отображающую основные сущности при работе автосалона. (обязательные объекты: Автомобиль, Ремонт при поломке, Клиент, Починка двигателя, Оплата услуг, Рабочий, Осмотр машины, Кассир, Проверка)

2. Создать диаграмму, отображающую основные сущности при работе кафе быстрого питания. (обязательные объекты: Кофеавтомат, Выдача товара, Клиент, Оформление заказа, Оплата услуг, Повар, Осмотр продуктов, Кассир)

3. Создать диаграмму, отображающую основные сущности при работе парикмахерской. (обязательные объекты: Уборщик, Стрижка, Клиент, Ожидание стрижки, Оплата услуг, Парикмахер, Установка требований, Кассир)

*От этого задания отойти нельзя.

Задание 2.

1. Вспомнить работу с графическим редактором Draw.io для создания алгоритмов и диаграмм.

2. Разработать диаграмму, согласно требованиям диаграммы и варианту задания.

Вариант.

1. Выполнить подробный анализ сказки «Репка».
2. Выполнить подробный анализ сказки «Курочка Ряба».
3. Выполнить подробный анализ сказки «Теремок».
4. Выполнить подробный анализ сказки «Гуси-Лебеди».
5. Выполнить подробный анализ сказки «Баба Яга».
6. Выполнить подробный анализ сказки «Колобок».
7. Выполнить подробный анализ сказки «Волк и семеро козлят».
8. Выполнить подробный анализ сказки «Три поросенка».
9. Выполнить подробный анализ сказки «Лиса и журавль».
10. Выполнить подробный анализ сказки «Кот, петух и лиса».
11. Выполнить подробный анализ сказки «Маша и медведь».
12. Выполнить подробный анализ сказки «Заюшкина избушка».
13. Выполнить подробный анализ сказки «По щучьему велению».
14. Выполнить подробный анализ сказки «Три медведя».
15. Выполнить подробный анализ сказки «Царевна лягушка».
16. Выполнить подробный анализ сказки «Гадкий утенок».

*Если кому-то не нравятся темы, придумайте свою и согласуйте с преподавателем.